

Question 3.1

Using the same data set (`credit_card_data.txt` or `credit_card_data-headers.txt`) as in Question 2.2, use the `ksvm` or `kknn` function to find a good classifier:

(a) using cross-validation (do this for the k-nearest-neighbors model; SVM is optional); and

Methodology

- I. To start off, I used `sample()` to select random indices of the dataset. I chose to select 70% of the data and store it as the training data. The remaining was put in the testing data.
- II. To perform cross-validation, I used `train.kknn()` where `kmax = 50` and `scale = TRUE`. After that, I created a variable for the training prediction to store a vector of 0's and initialized another variable for the training accuracy to 0.
- III. Then, I created a for loop that ran through all observations of the dataset, performed the `kknn()` function that utilized the training dataset, and set `k = 41`. I used `k = 41`. After the function, I took the integer of the fitted model plus 0.5 so that it could decide whether the value was 1 or 0. I calculated the accuracies of the training model by adding all the correct predictions and dividing it by the total number of observations of the training data.
- IV. For the testing data, I followed the same steps as the training data process, but instead, utilized the testing data, which covered a random 30% of the original data.

Results

The best `k` value was 41. I tested out `k = 41` for my training and testing models. The training data's accuracy percentage was 83.37%, and the testing data's accuracy percentage was 83.76%.

Discussion of Results

Based on these results of my `train.kknn()` code, it looked like 41 was a good value for `k`. The first reason was that having a training accuracy percentage between 80-90% was a good range. One worry I had was that the high `k` value may cause overfitting. However, looking at the accuracy percentage, it was around the same as the training accuracy percentage (83.37% to 83.76%). This proves that overfitting and underfitting did not exist. Instead, the accuracy percentage stayed consistent for both data. Therefore, I chose to stick with `k = 41` since there were no issues. I tried changing the `kmax` to see if there were any higher `k` values that displayed better results. However, the best `k` value stayed at `k = 41`. In context, the `k = 41` represents the 41 nearest credit card applications to one point. That one point will be classified as either accepted or rejected based on the scaled distance and classified points nearest to the point. The training accuracy percentage represents that the training data correctly classifies 83.37% of the credit card applications. This only covers 70% of the credit card applications. Moving on to the remaining 30%, the testing accuracy percentage represents that the testing data correctly classifies 83.76% of the credit card

applications as accepted or rejected. Therefore, I believe that splitting the dataset by 70-30 and using a k-value of 41 is a good classifier.

References:

<https://www.rdocumentation.org/packages/kknn/versions/1.4.0/topics/predict.train.kknn>

- (b) splitting the data into training, validation, and test data sets (pick either KNN or SVM; the other is optional).

Methodology

- I. I decided to split my data into 60% training, 20% validation, and 20% testing. To do this, I created a variable for my training index by creating a sample that took 60% of the data. Next, I created a variable for my remaining index by using `setdiff()` to get rid of the repeating indices and choose only the indices that have not been stored yet. Using that remaining index, I created the validation index by taking half of the remaining index so that it was 20% of the data. Then, I used `setdiff()` again, but with the validation index instead so that it took the other half (20%). Finally, I created the training, validation, and testing datasets by using those indices.
- II. I followed the same steps as the previous methodology, but I used a nested for loop. One for loop was so that I can display the plot of the accuracy percentages for each k value, and the second for loop was for the model I created using the `kknn()` function for the training dataset. I set the x-axis to k and y-axis to the accuracy percentage.
- III. For the validation and testing dataset, I followed the same procedure but just replaced the `kknn()` model's dataset to either validation or testing.

Results

Training data: The accuracy percentage was highest when k = 5, 6, 7, 8, and 21. The accuracy percentage was 85.45918%.

Validation data: The validation accuracy percentage was highest, valued at 83.20611% when k = 5.

Testing data: At k = 10, 11, and 12, the accuracy percentage was highest at 87.02290%. At k = 5, 6, 7, 8, and 21, the accuracy percentages ranged from approximately 83 – 86%. Specifically, at k = 5, the accuracy percentage was 83.96947%.

Discussion of Results

For the training data, at k = 5, 6, 7, 8, and 21, the training accuracy percentage represents that the training data (60%) correctly classifies 85.45918% of the credit card applications as accepted or rejected. For the validation data, I used 20% of the original data to evaluate the model's skill. The

highest validation accuracy percentage was 83.20611%, which is somewhat consistent compared to the training percentage. Therefore, I decided that it was a good model. Finally, for the testing data, at $k = 10, 11$, and 12 , the testing accuracy percentage represents that the testing data (last 20%) correctly classifies 87.02290% of the credit card applications as accepted or rejected. The testing model performed better than the training and validation models. The highest accuracy percentage is valued at 87.0229% when $k = 10, 11$, and 12 . However, if we look at the accuracy percentages at the validation k -values but for the testing data, at $k = 5$, the accuracy percentage is valued at 83.96947%. Although the accuracy percentage went down for $k = 5$, the accuracy percentage is still consistent and only differs by a small margin. Based on these results, I can confidently say that there was no overfitting or underfitting present since the testing accuracy percentage increased. This is an ideal situation, and it makes the model much more reliable. Therefore, I concluded that splitting the dataset by 60-20-20 presents a good classifier.

References:

<https://www.geeksforgeeks.org/r-language/get-exclusive-elements-between-two-objects-in-r-programming-setdiff-function/>

Question 4.1

Describe a situation or problem from your job, everyday life, current events, etc., for which a clustering model would be appropriate. List some (up to 5) predictors that you might use.

One situation I can think of where you would use clustering is game statistics. For example, in the game "League of Legends", let's say the response variable is "Win/Loss". Some predictors that we can consider for this would be "Number of Kills", "Amount of Gold", "Number of Jungle Objectives", "Players' Level", "Players' Rank", and many other things. We would perform clustering by finding out which predictors is the best combination and which k -value is best. From there, we would use that information to predict whether a team is going to win or lose.

Question 4.2

The *iris* data set iris.txt contains 150 data points, each with four predictor variables and one categorical response. The predictors are the width and length of the sepal and petal of flowers and the response is the type of flower. The data is available from the R library datasets and can be accessed with iris once the library is loaded. It is also available at the UCI Machine Learning Repository (<https://archive.ics.uci.edu/ml/datasets/Iris>). *The response values are only given to see how well a specific method performed and should not be used to build the model.*

Use the R function kmeans to cluster the points as well as possible. Report the best combination of predictors, your suggested value of k , and how well your best clustering predicts flower type.

Methodology

- I. The first thing I did was import the iris dataset and scale it.

- II. Next, I created four different variables, and each variable takes the kmeans of a specific number of centers. So, I tested 2, 3, 4, and 5 centers, and set `nstart = 20`. `Nstart = 20` means that `kmeans()` will run 20 times with a different random cluster centroid each time.
- III. Then, from the package “factoextra”, I used `fviz_cluster()` to plot the graphs of each model. Each cluster is color coded with its own shape and color.
- IV. I created a contingency table for each model to show where each species was classified in whichever cluster.
- V. Using `fviz_nbclust()`, I created the elbow plot. The x-axis represented the number of clusters `k` while the y-axis represented the total distance within the sum of squares.
- VI. After that, I calculated the accuracy for all the predictors in the model that used 3 clusters. To calculate the accuracy, I added the values of each species where its most populated (highest values of each) and divided it by the total (150).
- VII. Finally, I created a function that creates a contingency table for only 3 clusters, but I tested out different combinations of predictors. The predictors are sepal width, sepal length, petal length, and petal width. In other words, I would test out sepal width, sepal length, petal length as one combination and then test out all the different combinations. Then, I calculated the accuracy percentage for each combination.

Results

As I mentioned before, the best `k` value is 3. The best combination of predictors to use was sepal length, petal length, and petal width. The reason for this is that the accuracy percentage was highest for those three predictors, at 86.666667%.

Discussion of Results

Why did I decide 3 clusters is the best?

The reason for this is that in the contingency table with 3 clusters, all the setosa species are put in the first cluster, and versicolor and virginica roughly correspond with the other two clusters. When you look at the other contingency tables that use more clusters, it splits setosa into two different clusters. Additionally, the elbow plot's total distance seems to be minimal after `k = 3`. I concluded that 3 clusters are the best.

Why did I decide the best combination of predictors was sepal length, petal length, and petal width?

The reason for this is that the accuracy percentage was highest for those three predictors, at 86.666667%. This means that those three predictors differentiate setosa, versicolor, and virginica the best. Sepal Width creates the most noise, so it interferes with our `kmeans()` model.

References:

<https://www.rdocumentation.org/packages/stats/versions/3.6.2/topics/kmeans>

https://uc-r.github.io/kmeans_clustering

<https://www.geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k-in-kmeans/>

https://www.rdocumentation.org/packages/factoextra/versions/1.0.3/topics/fviz_cluster